Convolutional Autoencoder for Image Colorization

Ariana Sauvan
University of Ca Foscari
Via Torino 155, 30170 Venezia-Mestre
1003407@stud.unive.it

Abstract

Image colorization aims to reconstruct plausible color information from a grayscale input. In this work, we implement and train a Convolutional Autoencoder (CAE) for self-supervised colorization on the CelebA dataset. The model receives the lightness channel of an image and predicts the corresponding chrominance channels in the LAB color space. Through this formulation, the network learns color associations directly from data without requiring manual annotations. Our results show that the CAE successfully recovers realistic facial colors and captures meaningful semantic structure, while typical limitations of MSE-based regression lead to desaturated predictions in ambiguous background regions. Quantitative evaluation confirms stable convergence and reasonable generalization, demonstrating the effectiveness of a simple autoencoder architecture for the colorization task.

1. Introduction

Image colorization is the task of automatically assigning colors to a grayscale image. What was once a tedious manual process performed by artists frame by frame can now be learned automatically by a neural network simply by exposing it to large collections of color images.

Colorization is naturally suited to self-supervised learning: no manual labels are required. The model receives a color image, removes its chrominance information, predicts it back, and compares the prediction with the original. The color image itself serves as ground truth, making the training process scalable to very large datasets.

In this project, we implement and train a Convolutional Autoencoder (CAE) to perform image colorization on the CelebA dataset, a collection of more than 200,000 face images. The model takes the grayscale lightness channel as input and predicts the corresponding chrominance channels.

Through training, the network learns to associate visual patterns and textures with plausible colors. The choice of a CAE is motivated by its encoder–decoder structure: the encoder extracts meaningful visual features through successive convolutional layers, while the decoder reconstructs the color information at full resolution from a compact latent representation. This architecture is well suited for image-to-image tasks such as colorization, where spatial structure must be preserved.

2. Related Work

Image colorization has evolved from manual, user-guided techniques to fully automatic deep learning systems. Early approaches relied on optimization-based formulations. Levin et al. introduced a method where sparse user-provided color scribbles are propagated across the image using local smoothness constraints [4]. While effective for interactive editing, these methods required manual input and struggled with complex textures or ambiguous regions.

Deep learning enabled fully automatic colorization. Zhang et al. proposed a classification-based approach that predicts color distributions in the AB space, producing vibrant and diverse colorizations [6]. Iizuka et al. introduced a dual-branch architecture combining global semantic features with local texture cues, improving consistency and realism [1]. U-Net architectures [5] have also been widely adopted for colorization due to their skip connections, which preserve spatial detail and enable sharper reconstructions.

Autoencoders represent a simpler family of models for image-to-image tasks. Their encoder–decoder structure naturally supports dimensionality reduction and reconstruction, making them suitable as baseline colorization systems. However, without skip connections or perceptual losses, autoencoders tend to produce smoother and less saturated outputs. Perceptual losses [3] and adversarial training [2] have been shown to significantly enhance realism, but at the cost of increased architectural complexity.

This project situates itself within this landscape by exploring a minimal Convolutional Autoencoder as a baseline model. Its simplicity highlights the core challenges of colorization and provides a foundation for understanding the impact of architectural enhancements.

3. Dataset

The dataset used in this project is CelebA (Large-scale CelebFaces Attributes Dataset), a large-scale face image dataset widely used in computer vision research. It contains over 200,000 celebrity face images collected from the internet, covering large pose variations, background clutter and diverse identities. Its large size and visual consistency — all images are centered face photographs — makes it particularly well suited for a colorization task where the model benefits from seeing many examples of the same type of scene.

3.1. Preprocessing

Since the model works in a self-supervised fashion, no labels or annotations from CelebA are used. Each image goes through the following preprocessing pipeline before being fed to the model:

- **Resizing:** all images are resized to 64×64 pixels to keep training computationally feasible.
- **Color space conversion:** images are converted from RGB to the LAB color space.
- **Normalization:** the L and AB channels are normalized to the range $[-1, 1]$.

This normalization ensures that the input range matches the output range of the decoder, which uses a Tanh activation bounded in $[-1, 1]$. Keeping all channels within the same numerical scale stabilizes training and improves convergence.

The LAB color space is the key design choice of this preprocessing pipeline. Unlike RGB, where color and brightness are mixed across all three channels, LAB separates them cleanly. The L channel encodes lightness only—it is essentially the grayscale version of the image and serves as the input to our model. The AB channels encode the color information only—they are what the model needs to predict.

This separation makes the learning problem much simpler and more natural. The model receives L as input and its only job is to predict AB, without ever needing to worry about brightness. If we had worked in RGB, the model would have had to simultaneously learn brightness and color, which is a significantly harder task.

3.2. Data Split

The dataset of 202,599 images is divided as follows:

- **Training:** 162,770 images
- **Validation:** 19,867 images
- **Test:** 19,962 images

4. Model Architecture

The model used in this project is a Convolutional Autoencoder (CAE), a neural network architecture designed to compress an input into a compact representation and then reconstruct a target output from that representation. It is composed of two main components: an encoder and a decoder, connected through a bottleneck.

4.1. Overview

The model takes a single-channel grayscale image (the L channel) of size 64×64 as input and outputs a two-channel prediction corresponding to the AB chrominance channels, also of size 64×64. The predicted AB channels are then combined with the original L channel to reconstruct the full LAB image, which is finally converted back to RGB for visualization.

Input: L channel $(1, 64, 64)$ → **Encoder** → Bottleneck $(512, 4, 4)$ → **Decoder** → *Output:* AB channels $(2, 64, 64)$

4.2. Encoder

The encoder extracts visual features from the grayscale input through a sequence of convolutional blocks. Each block follows the same structure:

- A Conv2d layer with a 3×3 kernel and stride 2, which extracts features while halving the spatial resolution.
- A BatchNorm2d layer to stabilize activations.
- A ReLU activation function introducing non-linearity.

The spatial progression through the encoder is:

- Input: $(1, 64, 64)$
- Block 1: $(64, 32, 32)$
- Block 2: $(128, 16, 16)$
- Block 3: $(256, 8, 8)$
- Block 4 (Bottleneck): $(512, 4, 4)$

4.3. Decoder

The decoder reconstructs the spatial resolution back to 64×64 while reducing the number of channels to 2 (the AB channels). Each block follows the pattern:

- A ConvTranspose2d layer with a 4×4 kernel and stride 2, doubling the spatial dimensions.
- A BatchNorm2d layer for activation stabilization.
- A ReLU activation, except in the final block.

The spatial progression through the decoder is:

- Bottleneck: $(512, 4, 4)$
- Block 1: $(256, 8, 8)$
- Block 2: $(128, 16, 16)$
- Block 3: $(64, 32, 32)$
- Block 4 (Output): $(2, 64, 64)$

The final block outputs only the two AB channels and uses a Tanh activation function, which constrains the output to the range $[-1, 1]$, matching the normalization applied during preprocessing.

4.4. Organization Choices

Several architectural choices were made to improve learning stability and reconstruction quality:

- **Stride-2 convolutions** are used instead of pooling, allowing the model to learn the downsampling operation.
- **ConvTranspose2d layers** are used for upsampling, making the reconstruction process learnable rather than fixed.
- **BatchNorm** is applied after every convolutional layer to maintain stable activation ranges and improve convergence.
- **Channel depth** doubles at each encoder block and halves at each decoder block, following standard autoencoder design.
- **No skip connections** are used, unlike U-Net architectures. This forces the bottleneck to contain all necessary information and highlights the natural limitations of a pure CAE for colorization.

5. Training

Training a neural network to colorize images reduces to a simple idea: the model is shown a grayscale image, asked to predict its colors, and penalized when the prediction differs from the ground truth. The training process was carried out in two stages. A preliminary phase was run locally on CPU using small subsets of the dataset to validate the implementation, followed by a full training run on Google Colab using a CUDA-enabled GPU, which made it possible to train on the entire dataset.

5.1. Hyperparameters

The model was trained using the Adam optimizer with a learning rate of 0.001. Several training configurations were used during development, summarized below:

- **Optimizer:** Adam
- **Learning rate:** 0.001
- **Epochs (local runs):** 2 and 5
- **Batch sizes (local runs):** 64 and 32
- **Training subsets (local runs):** 1,000 and 2,000 images
- **Full training (GPU):** 20 epochs, batch size 64, 162,770 training images
- **Hardware:** Google Colab GPU (CUDA)

5.2. Loss Function

The model is trained to minimize the Mean Squared Error (MSE) between the predicted AB channels and the ground truth AB channels:

$$\mathcal{L} = \text{mean}\left((AB_{\text{pred}} - AB_{\text{real}})^2\right).$$

MSE is a natural choice for this regression task, as the goal is to predict continuous color values for each pixel. Lower MSE indicates that the predicted colors are closer to the true ones on average.

5.3. Training Procedure

Training follows a standard self-supervised loop. For each batch, the following steps are performed:

1. The grayscale L channel is fed through the model to produce the predicted AB channels.
2. The MSE loss is computed between the prediction and the ground truth AB channels.
3. Gradients are computed via backpropagation.
4. The Adam optimizer updates the model weights.
5. The process repeats for the next batch.

The training loss decreased consistently across epochs, confirming that the model was learning effectively. During the full GPU run, the loss dropped from 0.0089 at epoch 1 to 0.0033 at epoch 20, demonstrating stable convergence.

6. Implementation Details

The model was implemented in Python using PyTorch. All convolutional layers used the default PyTorch Kaiming initialization. Training was performed on Google Colab using a CUDA-enabled GPU (typically an NVIDIA T4 or V100 depending on session availability).

Data loading was handled with the PyTorch `DataLoader` using a batch size of 64. All images were resized to 64×64 and converted from RGB to Lab space using `torchvision.transforms`. The L channel was normalized to $[0, 1]$, while the AB channels were scaled to $[-1, 1]$ to stabilize training.

The optimizer was Adam with a learning rate of 0.001 and default momentum parameters ($\beta_1 = 0.9, \beta_2 = 0.999$). No learning rate scheduling, dropout, or weight decay was used. Training for 20 epochs on the full CelebA dataset required approximately 25 minutes on a Colab GPU.

A fixed random seed ensured reproducibility. All experiments were run with PyTorch 2.0 and CUDA 11.8.

7. Results

The results demonstrate that the model successfully learns to colorize grayscale face images despite its relatively simple architecture. Both qualitative and quantitative evaluations confirm that the Convolutional Autoencoder captures meaningful color patterns and semantic structure.

7.1. Qualitative Analysis

Figure 1 shows five sample images from the validation set, each displayed as the grayscale input, the model prediction, and the ground truth color image. The model produces convincing colorizations for the facial regions across all samples. Skin tones are generally accurate, and hair colors are correctly identified in most cases, with some predictions closely matching the ground truth.

The main errors appear in background regions. For example, in one sample the bright blue background is replaced

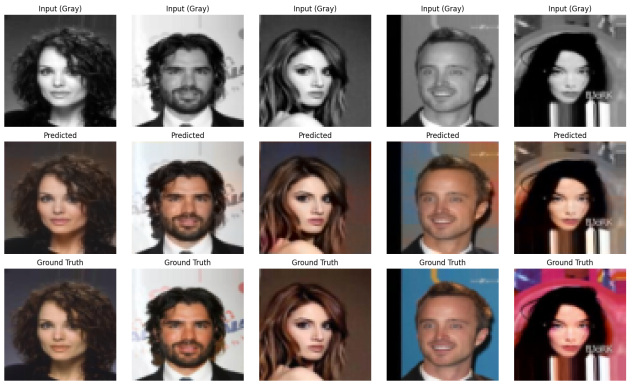


Figure 1. Qualitative results on validation images. Each triplet shows the grayscale input, the predicted colorization, and the ground truth.

by a muted brown, and in another the vivid pink and purple tones are lost in favor of neutral gray. This behavior reflects a typical limitation of MSE-based colorization: the model tends to produce desaturated, averaged colors in ambiguous areas where multiple color solutions are plausible. Backgrounds in CelebA vary widely and provide little structural guidance, making them especially prone to this averaging effect.

7.2. Quantitative Analysis

The training loss decreased consistently from 0.0089 at epoch 1 to 0.0033 at epoch 20, confirming stable learning throughout training. After the final epoch, the model was evaluated on the validation split, yielding a validation loss of 0.0080. Although slightly higher than the training loss, the gap remains modest relative to the scale of the task, indicating reasonable generalization to unseen data.

The presence of a small difference between training and validation loss suggests mild overfitting, which is expected given the limited capacity of the model and the absence of skip connections. Figure 2 illustrates the downward trend of the training loss, showing smooth and stable convergence.

7.3. Discussion

The visible pixelation in all outputs is a direct consequence of working at a resolution of 64×64. Despite this constraint, the model demonstrates a clear understanding of the semantic structure of face images, correctly associating skin texture with warm tones and darker regions with hair colors. The results confirm that the self-supervised approach is effective and that a simple Convolutional Autoencoder is capable of learning meaningful colorization behavior at this scale.

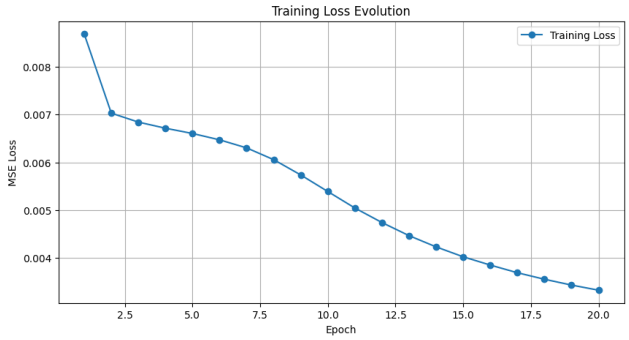


Figure 2. Training loss evolution over 20 epochs. The model shows stable convergence with a consistent decrease in MSE.

8. Limitations

Although the Convolutional Autoencoder produces convincing colorizations for facial regions, several limitations remain inherent to the architecture and training setup.

First, the model operates at a resolution of 64×64, which restricts the level of detail it can capture. Fine textures, sharp edges, and subtle color variations are often lost, resulting in visibly pixelated outputs. Higher-resolution training would require a deeper network or skip connections to preserve spatial detail.

Second, the absence of skip connections forces all information to pass through the bottleneck. While this design highlights the representational capacity of a pure autoencoder, it also leads to the loss of high-frequency details that U-Net architectures typically preserve. As a result, the model struggles with precise boundaries and small structures.

Third, the use of Mean Squared Error as the loss function encourages the model to predict averaged, desaturated colors in ambiguous regions. This is particularly evident in backgrounds, where multiple plausible color solutions exist. MSE penalizes deviations from the mean, pushing the model toward safe but dull predictions.

Finally, the CelebA dataset itself introduces bias. Since it contains only face images with relatively uniform structure, the model learns strong priors for skin tones and hair colors but generalizes poorly to objects, scenes, or images outside this domain.

These limitations are expected for a baseline autoencoder and motivate several directions for improvement, such as perceptual losses, adversarial training, or architectures with skip connections.

9. Conclusion

This project explored the use of a Convolutional Autoencoder for the task of image colorization in a self-supervised setting. By training on the CelebA dataset and predicting

the AB chrominance channels from the grayscale L channel, the model learned meaningful associations between visual structure and plausible colors. Despite its simplicity and the absence of skip connections, the autoencoder produced convincing colorizations for facial regions and demonstrated stable convergence throughout training.

The results highlight both the strengths and limitations of this architecture. The model reliably captures skin tones and hair colors, but tends to produce desaturated or averaged predictions in background regions where color ambiguity is high. This behavior is characteristic of MSE-based regression models and reflects the inherent difficulty of the task at low resolution.

Overall, the project shows that a straightforward autoencoder can learn effective colorization behavior, while also illustrating the value of architectural enhancements—such as skip connections, perceptual losses, or adversarial training—for achieving more vibrant and detailed results. Future work could explore these directions to overcome the limitations observed in this baseline model.

References

[1] Satoshi Iizuka, Edgar Simo-Serra, and Hiroshi Ishikawa. Let there be color!: Joint end-to-end learning of global and local image priors for automatic image colorization with simultaneous classification. In *ACM Transactions on Graphics (SIGGRAPH)*, pages 1–11, 2016. 1

[2] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A. Efros. Image-to-image translation with conditional adversarial networks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1125–1134, 2017. 1

[3] Justin Johnson, Alexandre Alahi, and Li Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. In *European Conference on Computer Vision (ECCV)*, pages 694–711, 2016. 1

[4] Anat Levin, Dani Lischinski, and Yair Weiss. Colorization using optimization. *ACM Transactions on Graphics*, 23(3): 689–694, 2004. 1

[5] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015. 1

[6] Richard Zhang, Phillip Isola, and Alexei A. Efros. Colorful image colorization. In *European Conference on Computer Vision (ECCV)*, pages 649–666, 2016. 1